TVCSI: Next Steps Plan

Response plan for the faculty review of the task-value-aware CSI feedback simulator

0. Quick glossary

Term	Meaning
gNB	The 5G base station (the tower).
Beam	The tower points its signal at a user like a flashlight. There are 32 fixed directions to choose from (the “codebook”).
CSI (Channel State Information)	The user telling the tower “this is what my signal looks like right now”. It's needed to pick the right beam.
Feedback mode	How detailed the CSI report is: mode 0 = no report (the tower guesses), modes 1/2/3 = 32/64/128 bits.
Budget	Total feedback bits allowed per time slot (256 bits for 8 users). Not everyone can report every slot.
SINR (γ)	Signal quality in dB. Higher is better.
Regret	How much signal you lose by using the wrong beam. 0 means the beam was perfect.
Q(γ)	“Task quality”: how well the message's meaning gets through at that SINR (0 to 1).
Critical user	A user whose messages must reach Q ≥ 0.8.
Age	Slots since a user last sent CSI. Higher age means the tower's information is staler.

The project in one line: each slot, choose which users send CSI and how detailed it is, so the important messages get through best without going over the bit budget.

1. What's going wrong (from our own results)

Problem	Evidence	Plain-English cause
A. Proposed loses to round-robin	Proposed 1.367 vs Baseline 1.601 utility. Ordering violated at 52 of 90 sweep points.	The tower's guess of “how much will feedback help this user?” is badly wrong: estimated gain ≈ 0.01, true gain ≈ 0.06, and the **correlation is ≈ 0**. The scheduler is basically choosing at random.
B. Mean-regret plug-in	`tvcsi.py:87-91` puts the **average** regret into Q	The beam either stayed put (0 loss) or moved (big loss). Averaging gives “always a small loss”, and because Q is S-shaped, a small loss looks harmless. So the tower thinks feedback is nearly worthless.
C. One-slot thinking	Greedy only looks at the current slot	A CSI report also helps the next several slots. Round-robin wins because it keeps **every user's age low**. Proposed lets some users go stale.
D. 97–98% critical violations	Even **Exact** (the cheating genie) violates 95.7%	Not mainly an algorithm bug. A critical user needs about 16.1 dB, but the median best-beam SINR is only about 11.3 dB (`ASSUMPTIONS.md` item 10). The target is set above what the radio can physically deliver. On top of that, no scheme tries to protect critical users (constraint 39j isn't implemented).
E. No real AI	`models/dummy.py`	The encoder adds random noise and the predictor repeats the last value. There's nothing to learn from.

Problem	Evidence	Plain-English cause
F. Arbitrary Q curve	`link.py:31`, `configs/default.yaml` γ0 = 14/10/6 dB	The logistic curve's numbers were made up, not measured.

2. The plan, in order

Each step lists **what** to do, **why**, **where**, and **how we'll know it worked**.

Phase 1: Measure properly first (sim.py)

Effort: about 2–3 days

This comes first because it tells us which failures are the algorithm's fault and which are physically impossible to avoid.

1. **Best-beam feasibility oracle.** Each slot, use the true channel to ask: “if this critical user got the perfect beam, could Q ≥ 0.8 be reached at all?” Log feasible[k,t].
2. **Split the violation metric** into (a) violations when the target was **feasible** (the scheduler's fault, and the number to report) and (b) violations when it was **infeasible** (physics, which no scheduler can fix).
3. **Add the missing constraints from the PDF as metrics:** reliability $\Pr[Q < Q_{min}] \le \varepsilon$ (39j), feedback penalty λ_F (39a), beam-switching penalty λ_B (39a), CVaR (39a), payload limit (39i).
4. **Keep the oracle evaluation-only.** The existing “tamper” test (spec test 10) must still pass, so true-channel data never reaches a decision.

Done when: the summary has a “feasible-only violation rate” column, and we know what fraction of the 97% is unavoidable.

Phase 2: Replace the made-up Q curve (link.py + config)

Effort: about 3–5 days

5. **Replace the logistic with a measured lookup table:** Q(SNR) points loaded from a CSV and interpolated. Two realistic sources: **Easier** – 5G NR link-level curves (block error rate vs SNR for a few MCS levels) from **Sionna** (NVIDIA's free 5G simulator, Python); use task success ≈ 1 − BLER. **More “semantic”** – train a small DeepJSCC model (or JPEG + channel code) on CIFAR-10 and measure **classification accuracy vs SNR**, one curve per message type (critical = stricter task).
6. **Recalibrate** SNR or thresholds so the critical target is **achievable most of the time with the right beam**. Aim for genie-feasible ≥ 90%. Write the reason in ASSUMPTIONS.md.

Done when: Q comes from a cited, reproducible measurement, and Exact's feasible-violation rate is low.

Phase 3: Fix the estimator (tvcsi.py) – the core algorithm fix

Effort: about 1 week

7. **Average the quality, not the regret.** Store the full regret distribution per bin (a histogram or quantiles), then compute `Uhat = v × mean over samples of Q(γ_ref × (1 − r))` instead of `Q(γ_ref × (1 − mean r))`. This directly fixes cause B.
8. **Give the estimator better clues than “age” alone:** age; **beam margin** (how close the best and second-best beams are – close means likely to flip); how far γ_{ref} is from the user's threshold (near the steep part of Q, small losses matter a lot); importance v; the size of the last surprise (how much the beam changed at the last report).
9. **Learn the value instead of using a lookup table.** Train a small model (gradient boosting or a tiny MLP) that predicts the **feedback gain** U[m] − U[0] from those features. This is the first real “AI” piece, and it answers the faculty's “why not AI”.

10 **Add future value (age-aware).** Value of feedback = gain now + discounted gain over the next few slots that the fresh CSI keeps helping. This fixes cause C.

Done when: the calibration table in the summary shows correlation of estimated vs true gain above 0.5 (currently ≈ 0), and **Optimised ≥ Proposed** (that ordering only holds once the estimate is good).

Phase 4: Make the algorithm protect critical users (solvers.py / schemes.py)

Effort: about 3–4 days

11 **Reliability "virtual queue"** (the standard Lyapunov method): each critical user keeps a counter that grows when they violate and shrinks when they don't. Add counter × Pr(violation) to their priority, so users who have been failing get pushed up automatically. This enforces 39j without changing the knapsack solver.

12 **Optional:** add λ_B (switching) and λ_F (bits) into the objective, as in Eq. 39a.

Done when: the feasible-violation rate for critical users is ≤ ε (for example 5–10%) while utility stays at or above the baselines.

Phase 5: Stronger, fairer baselines (schemes.py)

Effort: about 2 days

The faculty asked for these. Proposed must beat them **honestly**.

New baseline	Rule
Max-Age-First	Feed back the users with the stalest CSI.
Importance × Age	Priority = v × age.
Uncertainty-first	Feed back the users the predictor is least sure about (small beam margin or low confidence).
Feedback-gain-aware	Priority = v × expected Q loss if no feedback (a one-line heuristic).
Critical-first + round-robin	Critical users near their threshold first, then periodic for the rest.

Done when: every main figure shows Proposed against the best of these, with 95% confidence intervals.

Phase 6: Real trained AI models (models/)

Effort: about 1–2 weeks (can run in parallel with Phases 3–5)

The plug-in interface already exists (models/base.py), and scripts/export_dataset.py already builds train/val/test data.

13 **Beam predictor:** a GRU/LSTM that reads the last few CSI reports and predicts next-slot beam gains **plus an uncertainty** (an ensemble of 3–5 models or MC-dropout). That uncertainty replaces the fake exp(−age/τ) confidence and feeds straight into Phase 3.

14 **CSI encoder/decoder:** a CsiNet-style autoencoder with 3 compression levels (8/16/32 values × 4 bits). Train in PyTorch on the exported dataset.

15 Set models: trained in the config, refit the TVCSI estimator, and remove the "DUMMY MODELS" stamp.

Done when: the trained predictor beats persistence on beam accuracy on the test seeds, and figures are regenerated with real models.

Phase 7: Rerun and report

Effort: about 2–3 days

16 Run `bash scripts/run_all.sh`, regenerate all figures, and update summary.md and ASSUMPTIONS.md.

17 Write a short “what changed and why” note for the faculty that maps each of their comments to a fix and a figure.

3. Targets to show the faculty

- Proposed ≥ **every** implementable baseline at the default point, with the confidence interval excluding zero.
- Critical violation (feasible slots only) ≤ ε.
- Estimator correlation > 0.5, and Exact ≥ Optimised ≥ Proposed.
- Trained models, measured Q curve, no dummy stamp.

One rule: don't weaken baselines or tune until Proposed wins. Faculty spot that quickly. If Proposed still loses somewhere, report it and explain why.

4. Questions to ask the faculty

1 What violation tolerance ε should critical users meet (for example 5% or 10%)?

2 Which “practically measured” Q source would they accept: 5G NR BLER curves, or a semantic task such as image classification accuracy?

3 Should the channel stay synthetic, or move to realistic data (DeepMIMO or 3GPP CDL via Sionna)? The DeepMIMO loader is currently a stub.

5. Where to start

Phase 1 (the oracle) plus step 7 (averaging quality instead of regret). Both are small code changes, and they'll show quickly how much of the gap is fixable.